Running Pattern Analysis (RPA)

Classifying Foot Strike Patterns from Video Using VideoMAE

Nir Gofman, Gavan Vainberger

Computer Science, Technion

March 2026

Abstract

This project develops an end-to-end pipeline for classifying running foot strike patterns (forefoot vs. heel strike) from trackside video using VideoMAE, a video masked autoencoder pre-trained on Kinetics-400. The pipeline integrates YOLOv8-Pose for person detection, multi-stage filtering to isolate runners, automated clip extraction, and systematic data augmentation. Through 30 experiments testing different augmentation combinations, we found that brightness/contrast augmentation alone achieves the best test accuracy of 76.7%—a +23.6% improvement over the 53.1% baseline. Applying all augmentations simultaneously ranked only 9th (68.9%), demonstrating that targeted augmentation outperforms brute-force approaches for fine-grained biomechanical classification.

Introduction

Biomechanical analysis of running gait is critical for coaching, injury prevention, and rehabilitation. Identifying the foot strike pattern—forefoot vs. heel—is traditionally done manually by experts reviewing video frame by frame: labour-intensive, subjective, and unscalable.

This project automates foot strike classification from **standard trackside video** using deep learning. Rather than specialised motion-capture equipment, the system works with ordinary camera footage. The core research question: *can a video classification model, trained on short clips of a runner's foot region, distinguish between forefoot and heel strike patterns?*

The project addresses three challenges: (1) **data preparation**—processing noisy multi-person video into clean foot-region clips; (2) **model training**—adapting a pre-trained video transformer with limited data (17 runners); and (3) **augmentation strategy**—determining which augmentations help vs. hurt performance.

Initial assumption.
A natural assumption going into this study was that applying more data augmentations simultaneously would lead to better generalisation—after all, more variety in training data should make the model more robust. We initially expected the configuration using all augmentation types together to achieve the highest accuracy. As the results in Section 7 will show, this assumption turned out to be wrong: simpler, targeted augmentation strategies consistently outperformed complex multi-augmentation approaches.

Related Work

VideoMAE [1] introduces masked autoencoding for video transformers, exploiting temporal redundancy to mask 90–95% of space-time patches via tube masking. Pre-trained on Kinetics-400 (240K clips, 400 classes), the ViT-Base architecture has 87M parameters, 12 transformer layers, and 768-dimensional embeddings. The Vision Transformer (ViT) [2] provides the architectural foundation, dividing input into fixed-size patches

processed by a standard Transformer encoder. We fine-tune from the **MCG-NJU/videomae-base-finetuned-kinetics** [4] checkpoint (86.5M params, 80.9% top-1 on Kinetics-400). The full architecture is illustrated in Figure 1.

YOLOv8-Pose. YOLO (You Only Look Once) [3] is a family of real-time object detection models that frame detection as a single regression problem, predicting bounding boxes and class probabilities directly from full images in one forward pass. YOLOv8, the latest generation by Ultralytics, extends this with a pose estimation variant—**YOLOv8-Pose**—which simultaneously performs person detection and 17-point COCO keypoint estimation. The keypoints cover the full body skeleton, including critical anatomical landmarks such as the left and right ankles (indices 15 and 16). In our pipeline, YOLOv8-Pose serves as the entry point: it detects all persons in each frame, provides bounding boxes for spatial localisation, and extracts ankle coordinates that drive both the runner filtering heuristics and the foot-region ROI (Region of interest) calculation. We use the nano variant (`yolov8n-pose.pt`) for its optimal speed–accuracy tradeoff, enabling batch processing of hundreds of 30-second video clips at 60fps.

Foot Strike Patterns in Distance Running. Moore et al. [5] present a systematic review and meta-analysis of foot strike patterns during overground distance running, synthesising findings on prevalence, performance, and pattern changes across distance. Their meta-analysis reveals that 79% of long-distance runners use a rearfoot (heel) strike early in a race, with prevalence rising to 86% as distance increases. Of the 11% of runners who change pattern mid-race, the vast majority (84%) transition from non-rearfoot to rearfoot strike. These findings establish that foot strike classification is not merely an academic exercise—it captures a fundamental biomechanical variable that shifts dynamically during competition and has been investigated for potential performance implications, though the evidence for a competitive advantage of either pattern remains inconclusive. Our project automates this classification from video, which could enable large-scale studies of foot strike prevalence and transitions that are currently limited by the labour-intensive nature of manual annotation.

Biomechanical Importance of Foot Strike Classification. Almeida, Davis, and Lopes [7] conducted a systematic review with meta-analysis of 16 studies (1,260 runners) comparing the biomechanics of rearfoot, midfoot, and forefoot strike patterns. Their meta-analysis found that rearfoot strikers exhibit significantly higher vertical loading rates—a variable linked to tibial stress fractures—while forefoot strikers impose greater loads on the Achilles tendon and ankle joint. Each pattern produces distinct kinematic profiles at initial contact. The authors conclude that clinicians must be aware of these pattern-specific characteristics to guide injury management and training decisions. This work provides the biomechanical motivation for our project: while Almeida et al. establish *why* accurate foot strike classification is a clinical prerequisite, our system addresses *how* to perform it automatically from video at scale.

Data Augmentation. Mumuni & Mumuni [6] provide a comprehensive survey of modern data augmentation approaches for deep learning, covering geometric transforms (rotation, flipping, scaling), photometric transforms (brightness, contrast, colour jittering), and occlusion-based methods (cutout, random erasing). Their work highlights that augmentation effectiveness is highly task-dependent—transforms that help in one domain may hurt in another. This insight directly motivated our systematic augmentation study, where we test each augmentation type in isolation and in combination rather than applying all transforms blindly.

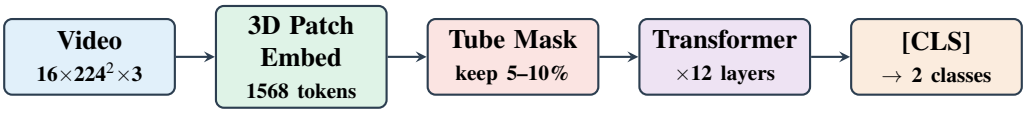

Figure 1: VideoMAE architecture. Input video is patchified, 90–95% masked, encoded by 12 transformer layers, and classified via the [CLS] token. We replace the pre-trained Kinetics head with a 2-class linear layer.

Pipeline Description

The system is organised as a six-phase pipeline, shown in Figure 2. Phases 1–3 handle data preparation—from raw video to a clean, split dataset—while Phases 4–6 cover the experiment pipeline: augmentation, training, and evaluation.

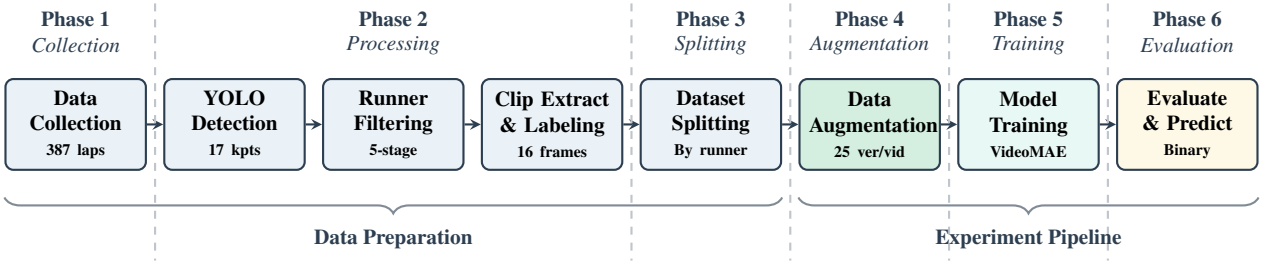

Figure 2: End-to-end pipeline. Phases 1–3 handle data preparation; Phases 4–6 cover the experiment pipeline. Dashed separators mark phase boundaries.

Phase 1: Data Collection

The raw data consists of **387 lap recordings** of 17 unique runners, filmed at 60fps from trackside cameras. Each recording captures approximately 30 seconds of a single lap and contains the full trackside scene—runners, spectators, officials, and varying backgrounds. The recordings are organised across 39 session directories following a consistent naming convention: `{RunnerID}_{RUN#}_{CAM#}/clips/lap_{NNN}.mp4`. For example, `04HN_RUN2_CAM1/clips/lap_006.mp4` refers to runner 04HN, second session, camera 1, lap 6. Multiple cameras (CAM1, CAM2, CAM3) capture the same running events from different angles along the track, providing diverse viewpoints.

Phase 2: Video Processing & Labeling

From the 387 raw recordings, the video processing pipeline extracts clean clips of individual runners' foot regions, producing the final dataset of **3,022 clips**.

YOLO Detection. YOLOv8-Pose detects all persons per frame with confidence >0.25, extracting 17 keypoints including ankles (indices 15, 16).

Runner Filtering. A multi-stage pipeline isolates runners from spectators and officials, reducing 8–15 initial detections to 1–2 validated runners per video. Developing robust filtering heuristics was one of the most challenging aspects of the project, as the fundamental tradeoff—too aggressive filtering loses real runners, while too lenient filtering trains on noise—could not be resolved analytically and demanded iterative refinement. The five successive filter stages are illustrated in Figure 3.

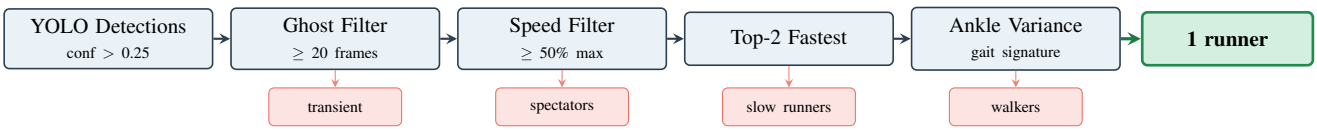

Figure 3: Runner filtering pipeline. Five successive filters progressively isolate target runners from non-runner detections. Red boxes show rejected categories at each stage.

Trackside videos typically contain 8–15 detected persons per clip, including spectators standing at the sideline, officials walking across the track, and camera operators. The filtering pipeline applies five successive stages to isolate the actual runners:

Ghost track filter. Removes transient detections with fewer than 20 frames. These arise from brief oc-

clusions, people momentarily entering the frame edge, or fragmented YOLO tracks that split and rejoin. Requiring a minimum track length ensures only sustained presences are considered.

Speed filter. Retains only tracks moving at $\geq$50% of the maximum observed speed in the video. This immediately removes stationary spectators and slow-moving officials, as their pixel displacement per frame is far below that of an active runner.

Top-2 selection. Keeps only the two fastest remaining tracks, targeting the primary subjects of the recording. Most trackside clips capture one or two runners passing the camera; additional detections at this stage are typically noise.

Ankle variance filter. Checks for the rhythmic vertical oscillation characteristic of running gait. High ankle variance indicates a running stride—feet moving up and down in a cyclical pattern—while low variance indicates standing or walking. This is the most discriminative filter, as it directly measures the biomechanical signature of running.

Clip Extract & Labeling. Once runners are isolated, foot-region crops (ROI) are sized at 40% of bounding box height, with ankles at 35% from the top. Stride-aware sizing widens the crop for side views. A 15-frame moving average prevents jitter, producing 224$\times$224 grayscale crops. VideoMAE requires exactly 16 frames; at 60fps this spans $\approx$267ms—one complete foot strike cycle. Clips use a sliding window with stride 16 (no overlap). The recompression is **lossless**: frames are written as PNG, then encoded with H.264 CRF 0 and YUV444p, preserving full visual fidelity.

After clip extraction, each of the 3,022 clips was labeled by biomechanics experts as Class 000 (flat foot), 001 (heel strike), or 002 (toes first). The distribution across the three original classes is shown in Figure 4.

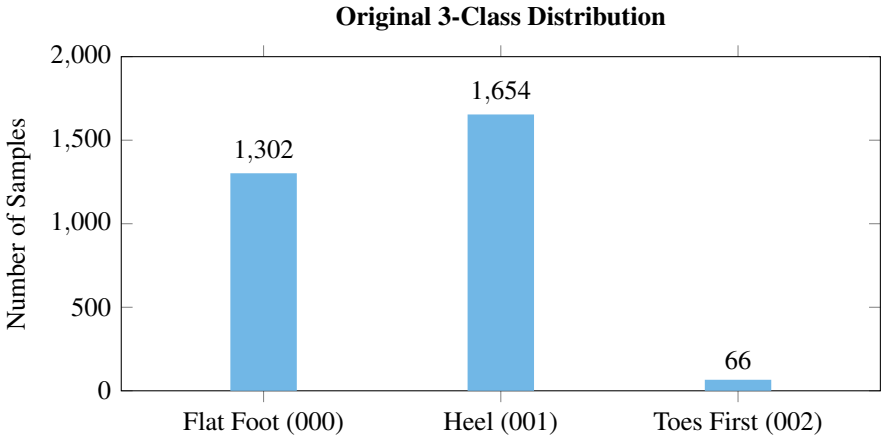

Figure 4: Original 3-class distribution. Class 002 (toes first) has only 66 samples (2.2%)—far too few for training. Since toes-first landing is biomechanically similar to flat-foot contact, we merged 002 into 000 for binary classification.

Class 002 (toes first) had only 66 samples (2.2%), making it impractical to learn as a separate class. Since landing on the toes is biomechanically closer to a flat-foot strike than to a heel strike, we merged Class 002 into Class 000, pivoting to **binary classification**: forefoot (flat foot + toes first) vs. heel strike.

Phase 3: Dataset Splitting

We split by runner to prevent data leakage—the same runner never appears in both training and test sets. An earlier random split had a 23.1% val/test distribution gap; the runner-based split reduces this to 4.7%. The resulting class distribution across all three splits is shown in Figure 5.

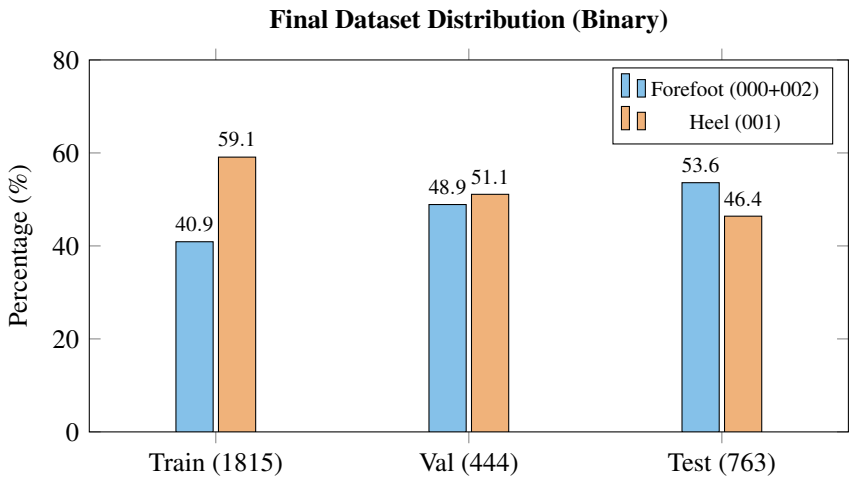

Figure 5: Binary class distribution per split after merging Class 002 into Class 000 and runner-based splitting.

Phase 4: Data Augmentation

Augmentation is performed **offline** before training. Each training video is augmented into **25 versions** with deterministic seeding (MD5 hash of path + version) for reproducibility. Seven augmentation types are used:

Table 1: Augmentation types

Augmentation	**Key Parameters**	**Prob.**
Grayscale	Always applied first	100%
Horizontal Flip	Mirror left-right	70%
Brightness/Contrast	$\pm$25% each	80%
Gaussian Blur	σ: 0.1–1.0	60%
Rotation	$\pm 5^\circ$	80%
Scale/Crop	0.9–1.1$\times$	60%
Cutout	15% area, upper 50% only	60%

A critical design choice: **Cutout is restricted to the upper 50% of the frame**, preserving the foot-level signal while forcing the model to not rely on upper-body context. The visual effect of different augmentation types on the foot-region crops is shown in Figures 6, 8, and 9.

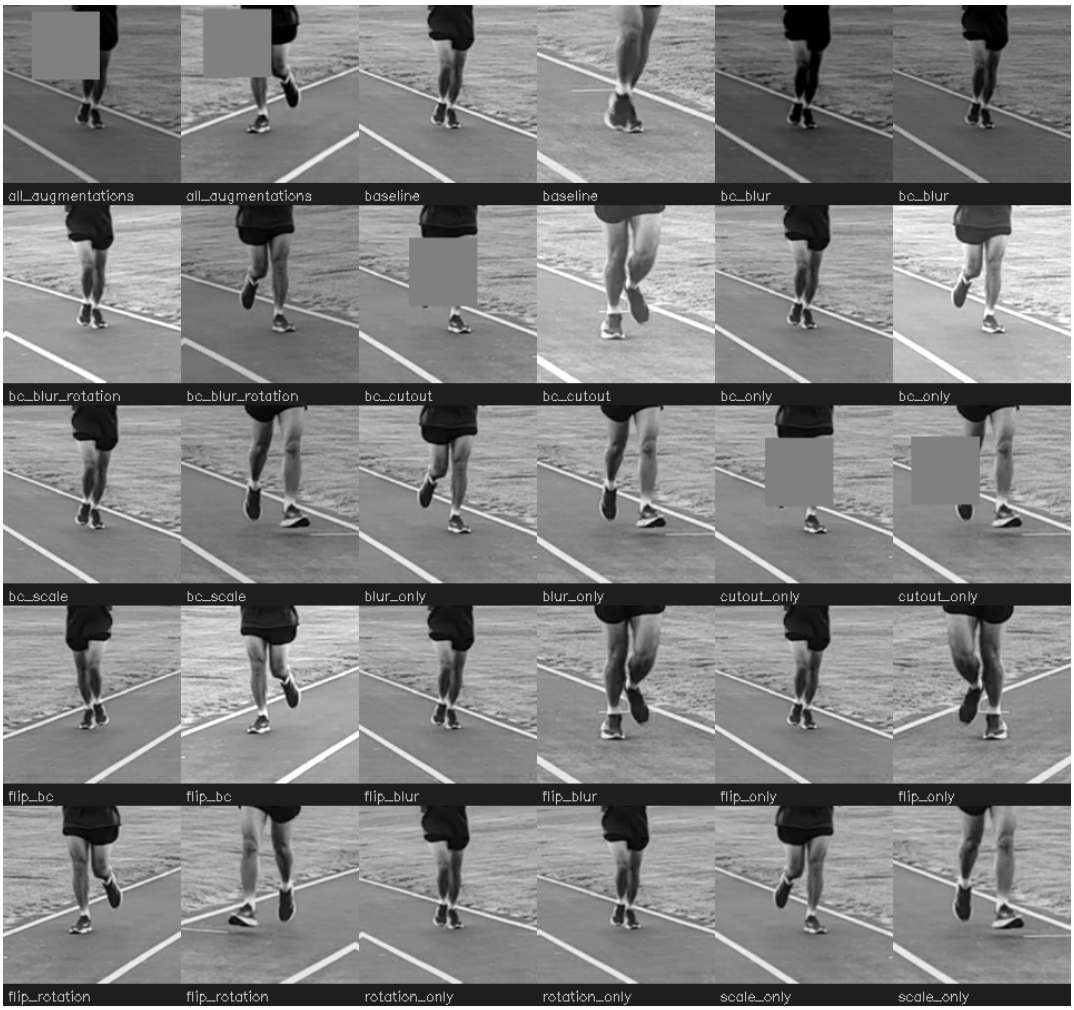

Figure 6: 30 augmented examples from various experiment configurations, showing the visual effect of different augmentation types and combinations on foot-region crops.

(a) Baseline (b) B/C only (#1) (c) Cutout only (d) All aug. (#9)

Figure 7: Augmentation examples on 224×224 grayscale foot-region crops (runner 04HN).

(a) Baseline (b) B/C only (#1) (c) Cutout only (d) All aug. (#9)

Figure 8: Augmentation examples on a different runner (27BY). Cutout clearly masks the upper region while preserving feet. All-augmentations combines cutout, blur, and rotation simultaneously.

Phase 5: Model Training

We fine-tune **MCG-NJU/videomae-base-finetuned-kinetics**, replacing the pre-trained 400-class Kinetics head with a new 2-class linear layer (forefoot vs. heel strike). The pre-trained encoder weights are kept and jointly fine-tuned alongside the new head.

Optimizer and learning rate. We use AdamW with a learning rate of 5×10^{-5}. The low learning rate ensures the pre-trained representations are adjusted gradually rather than overwritten.

Batch size. Each training batch contains 8 video clips. This is constrained by GPU memory—each clip is a 16-frame $\times$ 3-channel $\times$ 224 $\times$ 224 tensor (approximately 14MB per clip), and the 87M-parameter model requires substantial memory for forward and backward passes. A batch size of 8 balances memory utilisation with gradient stability.

Cross-entropy loss. We use cross-entropy as the loss function because it is the standard choice for classification tasks. It measures the divergence between the model's predicted probability distribution (softmax output) and the true label, penalising confident wrong predictions more heavily than uncertain ones. For our binary classification, cross-entropy provides well-calibrated gradients that drive the model toward the correct class.

ImageNet normalization. Each input frame is normalised using ImageNet channel-wise statistics (mean $= [0.485, 0.456, 0.406]$, std $= [0.229, 0.224, 0.225]$). This normalization is applied because the VideoMAE encoder was originally pre-trained on data normalised with these values. Using the same statistics ensures the input distribution matches what the pre-trained weights expect, preserving the learned feature representations.

Training configuration. Training runs for 10 epochs with no online augmentation—all augmentation is applied offline before training begins (Phase 4). The best model checkpoint is selected by validation accuracy. Device selection follows a priority cascade: CUDA $>$ MPS (Apple Silicon) $>$ CPU.

Phase 6: Evaluation & Prediction

At inference time, the pipeline loads the best model checkpoint (selected by validation accuracy during training) and processes input video through the same preprocessing pipeline: frame loading, grayscale conversion, temporal sampling to 16 frames, and ImageNet normalization. The model outputs a predicted class label (forefoot or heel strike) and a confidence score derived from the softmax probability. This enables both batch evaluation on the held-out test set and single-clip inference for new videos.

Engineering Tradeoffs

Runner filtering thresholds required extensive empirical tuning. Too aggressive filtering loses real runners (false negatives); too lenient keeps noise that degrades training data. The speed ratio, ankle variance, and minimum frame thresholds were iteratively refined on representative videos.

ROI stabilisation. YOLO bounding boxes jitter frame-to-frame. Without the 15-frame height smoothing, cropped regions "pulse"—zooming in and out—confusing the classifier. Smaller windows still show jitter; larger windows lag behind genuine height changes.

Data leakage. The initial random split mixed clips from the same runner across train/test, inflating accuracy via memorised runner-specific features. Switching to runner-based splitting caused an apparent performance drop but produced honest results.

Label imbalance. Class 002 ("Toes first") had only 66/3,022 samples. Merging 002$\rightarrow$000 treats toes-first and flat-foot landings as a single forefoot class, yielding a viable binary problem.

Experiment Infrastructure

Running 30 experiments—each augmenting 1,815 videos into 25 versions followed by GPU training—demanded production-grade infrastructure. The total compute exceeded **270 GPU-hours**, producing **≈1.36M augmented files** (≈400 GB on GCS), as broken down in Figure 10. The experiment lifecycle—from configuration through augmentation, training, and result tracking—is shown in Figure 11.

Figure 9: Scale of the experiment infrastructure.

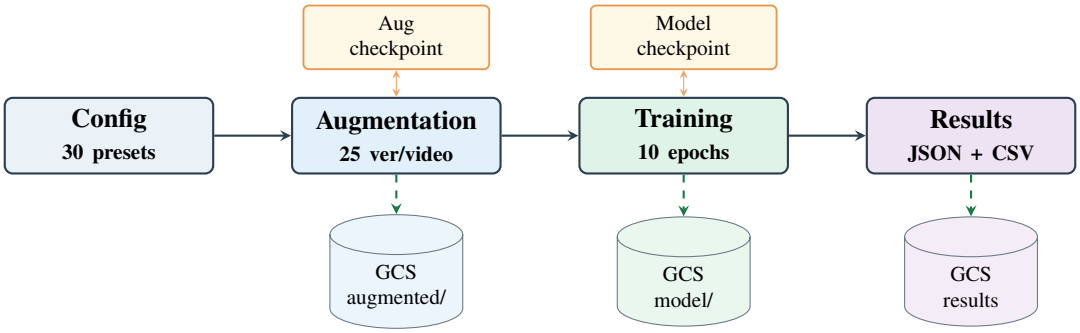

Figure 10: Experiment lifecycle. Data flows to GCS (dashed); checkpoints (orange) enable resumption at each stage.

Key infrastructure features:

Two-phase pipeline. Each experiment has an independent augmentation phase and training phase. Phases can be batched and re-run independently—e.g., retrain without re-augmenting.

Three-level checkpointing. (1) Suite-level checkpoint tracks which experiments are complete; (2) per-experiment checkpoint tracks which videos are augmented; (3) model checkpoints save best weights.

Race-condition-safe parallel execution. Multiple runners execute on different GPU nodes concurrently. A merge-based strategy (read → set-union → write) prevents data loss from concurrent writes to shared GCS state.

GCS integration. All data is stored in Google Cloud Storage with structured directories per experiment. An abstraction layer we developed enables transparent local/cloud operation.

Augmentation Study

We designed 30 experiments: 1 baseline, 6 single augmentations, 12 pairs, 10 triples, and 1 all-augmentations. All share identical training configuration to ensure controlled comparison.

Results

The complete results are listed in Table 2. Figure 14 ranks all 30 experiments by test accuracy, while Figure 12 compares the average performance across augmentation categories. Figure 13 isolates the six single-augmentation experiments to highlight the dominance of brightness/contrast.

Table 2: All 30 experiments ranked by test accuracy

Rk	Experiment	Augmentation(s)	Train	Test
1	bc_only	B/C	98.7%	**76.7%**
2	bc_cutout	B/C + Cutout	99.5%	75.1%
3	bc_scale	B/C + Scale	98.4%	73.9%
4	flip_rotation	Flip + Rotation	98.5%	71.7%
5	bc_blur_rotation	B/C + Blur + Rot	99.4%	70.5%
6	flip_blur	Flip + Blur	98.5%	70.2%
7	flip_bc_blur	Flip + B/C + Blur	98.4%	69.9%
8	blur_rotation	Blur + Rotation	98.7%	69.5%
9	all_augmentations	All 6	99.3%	68.9%
10	bc_blur_cutout	B/C + Blur + Cutout	98.6%	68.5%
11	flip_scale	Flip + Scale	98.4%	68.3%
12	cutout_only	Cutout	99.7%	68.2%
13	bc_blur	B/C + Blur	98.7%	68.0%
14	blur_rot_scale	Blur + Rot + Scale	98.4%	67.9%
15	flip_bc	Flip + B/C	98.5%	67.6%
16	bc_rot_scale	B/C + Rot + Scale	99.4%	67.4%
17	flip_only	Flip	98.7%	65.9%
18	rotation_only	Rotation	98.5%	65.7%
19	flip_rot_scale	Flip + Rot + Scale	98.2%	65.5%
20	flip_cutout	Flip + Cutout	98.7%	65.4%
21	bc_rotation	B/C + Rotation	99.5%	64.6%
22	rot_scale_cutout	Rot + Scale + Cut	98.5%	61.6%
23	scale_only	Scale	98.6%	61.2%
24	flip_blur_cutout	Flip + Blur + Cut	98.5%	60.7%
25	rotation_cutout	Rot + Cutout	98.5%	60.4%
26	flip_bc_cutout	Flip + B/C + Cut	98.4%	60.3%
27	rotation_scale	Rot + Scale	98.5%	60.0%
28	blur_only	Blur	98.6%	59.1%
29	flip_bc_rotation	Flip + B/C + Rot	98.0%	57.1%
30	baseline	None	91.6%	53.1%

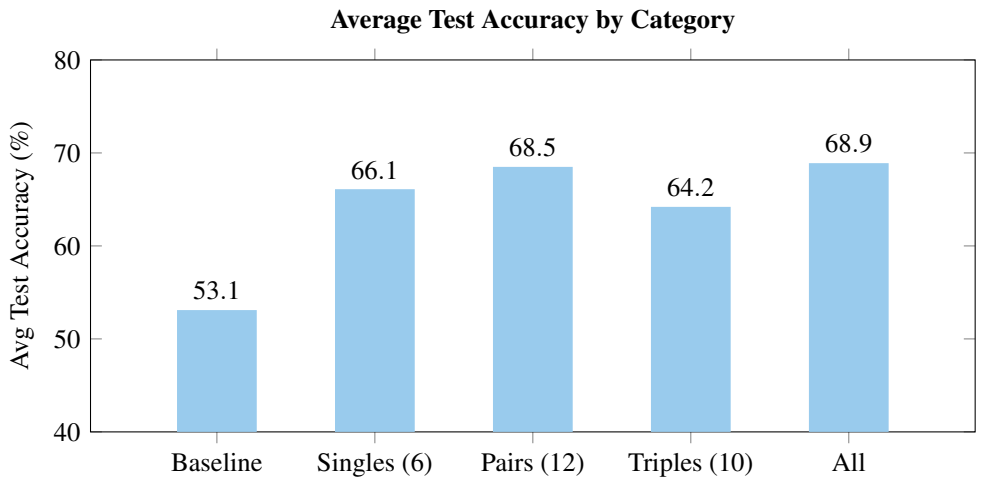

Figure 11: Triples (64.2%) perform *worse* than singles (66.1%) and pairs (68.5%). More augmentations $\neq$ better.

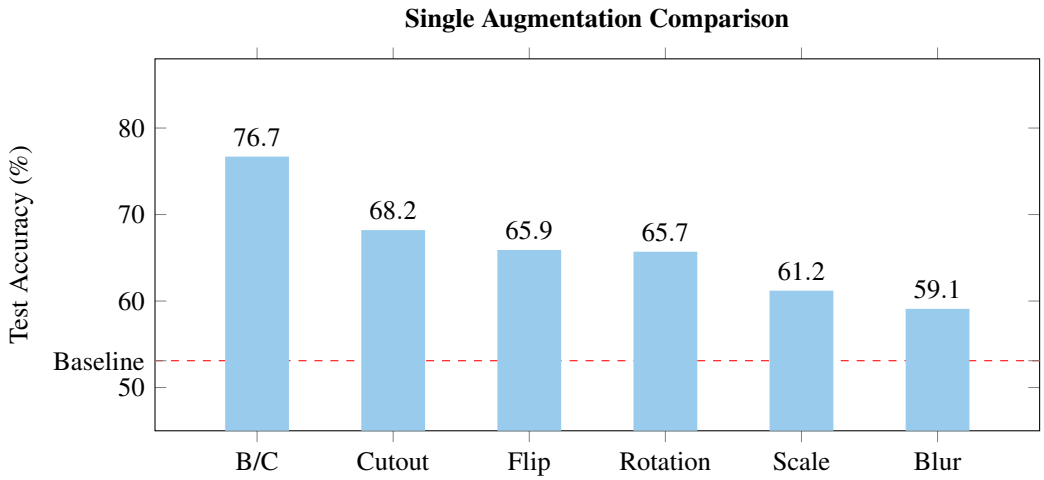

Figure 12: B/C (76.7%) dramatically outperforms all others. Blur (59.1%) barely improves on baseline.

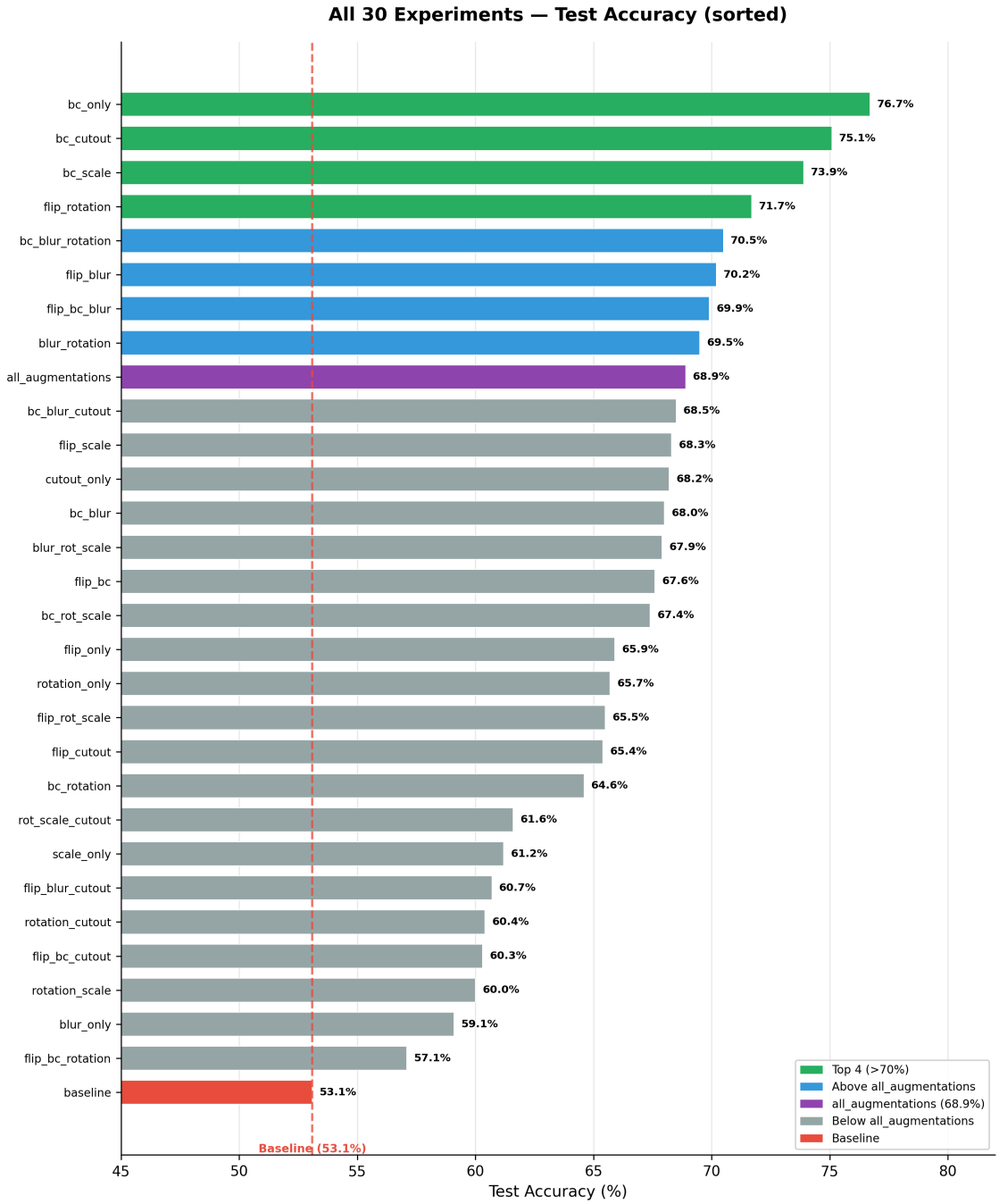

Figure 13: Test accuracy for all 30 experiments. Green: top 4 (>70%), Blue: above all_augmentations, Purple: all_augmentations itself (68.9%), Gray: below, Red: baseline. Dashed line marks the baseline (53.1%).

Key Findings: More Augmentations Does Not Mean Better Results

Applying all augmentations together ranked only 9th out of 30 experiments with 68.9% test accuracy, outperformed by 8 simpler configurations. The best result came from using brightness and contrast adjustment alone, achieving 76.7%—a single augmentation beating every multi-augmentation combination.

The top 3 experiments all include brightness/contrast: bc_only (76.7%), bc_cutout (75.1%), bc_scale (73.9%). Why?

- **Spatial distortion destroys subtle features.** Foot strike is encoded in subtle pixel arrangements—foot angle, heel position, sole shape. Rotation and scale distort these features.

- **Excessive variation overwhelms learning.** Multiple simultaneous transforms create too much variation for the model to learn from 1,815 clips.
- **B/C is domain-appropriate.** It simulates real lighting variation (shadows, exposure) without distorting spatial structure—preserving the features that matter.

Despite augmentation gains, all experiments exhibit severe overfitting: training accuracy exceeds 98% while test accuracy ranges from 53% to 77%, as shown in Figure 15. This ∼22% gap is likely due to the limited training diversity of only 4 runners.

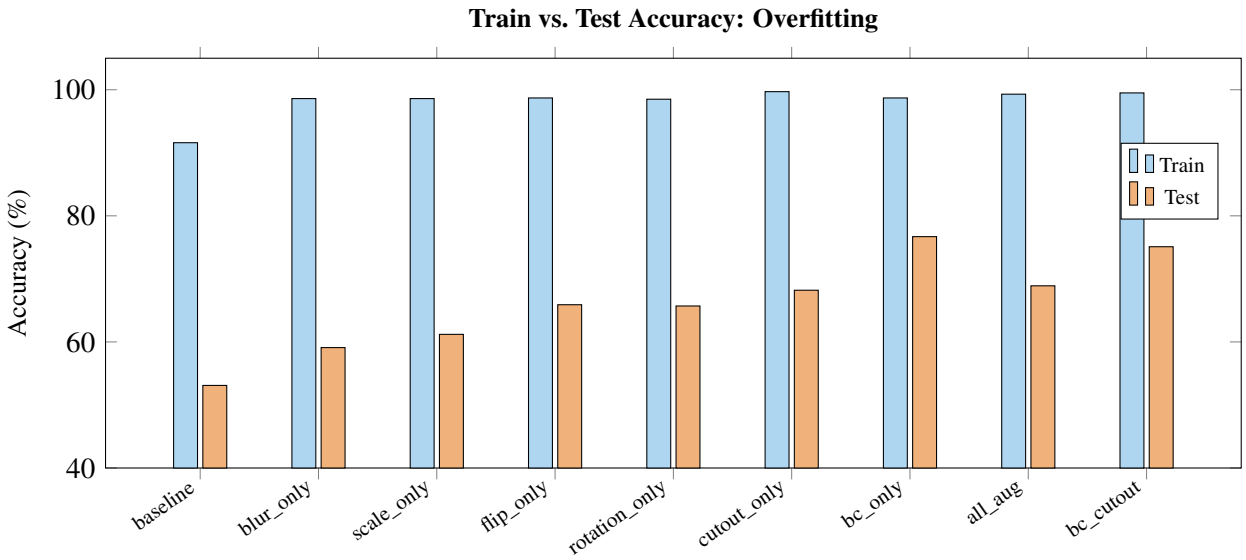

Figure 14: All experiments show >98% train accuracy but 53–77% test accuracy—severe overfitting from limited training diversity (4 runners).

To illustrate the training dynamics in detail, Figures 16 and 17 show the per-batch loss and accuracy convergence for the bc_only experiment. The training loss drops rapidly within the first few thousand batches and approaches zero, while training accuracy climbs above 99%. The validation accuracy markers reveal the gap between training and generalisation performance.

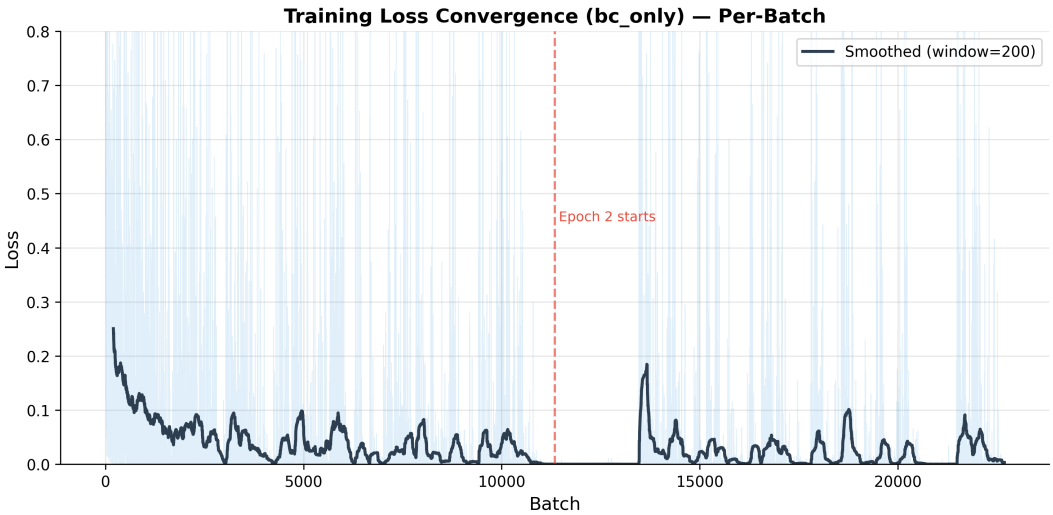

Figure 15: Training loss convergence for the bc_only experiment, showing per-batch loss across 2 epochs (≈22,700 batches). The smoothed curve shows rapid convergence within the first 2,000 batches, with loss approaching zero by epoch 2.

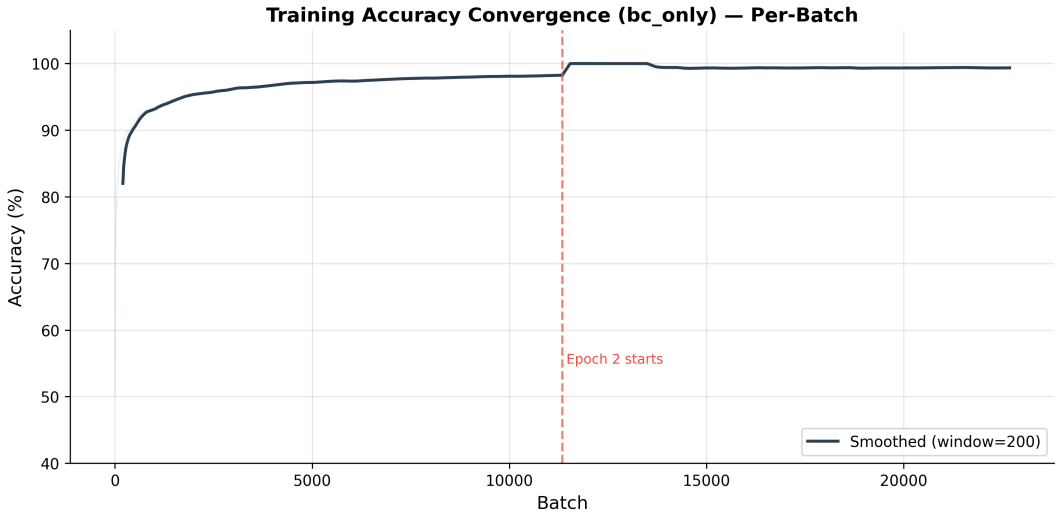

Figure 16: Training accuracy convergence for the bc_only experiment, showing per-batch running accuracy with validation accuracy markers at epoch boundaries. Training accuracy reaches >99% while validation remains at ∼50%, reflecting severe overfitting.

Figure 18 shows the confusion matrix for the bc_only model on the test set. The model achieves 75.4% recall on forefoot and 61.9% recall on heel strike, with an overall accuracy of 68.0%.

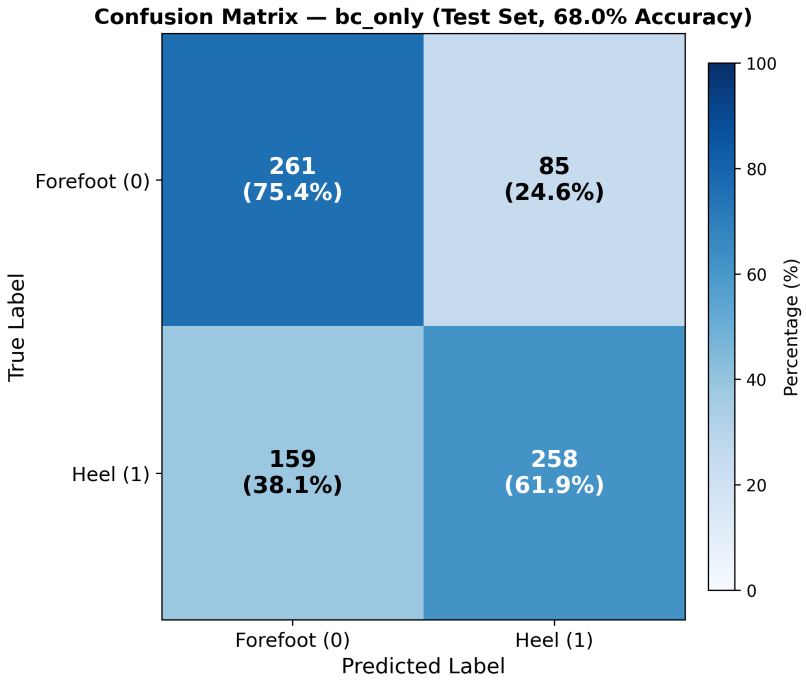

Figure 17: Confusion matrix for the bc_only experiment on the test set (763 clips). The model correctly classifies 261/346 forefoot clips (75.4%) and 258/417 heel clips (61.9%), for an overall accuracy of 68.0%.

Conclusions & Future Work

Key Findings

1. **B/C is the most impactful augmentation**—76.7% test accuracy, +23.6% over baseline.
2. **Simple beats complex**—top 3 all involve B/C with ≤1 additional augmentation. Triples average worse

than pairs.

3. **Severe overfitting**—∼22% train-test gap due to only 4 training runners.
4. **Runner-based splitting prevents leakage**—essential for honest evaluation.
5. **16 frames ≈ 1 foot strike cycle at 60fps**—natural temporal alignment with VideoMAE's input.

The diminishing returns with augmentation complexity are quantified in Figure 19: the best single augmentation improves over baseline by +23.6%, while the best triple achieves only +17.4%.

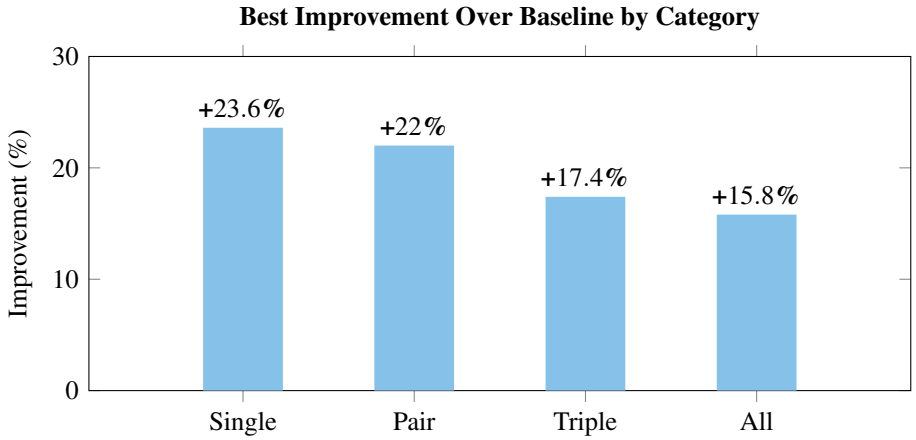

Figure 18: Diminishing returns with complexity. Best single (+23.6%) outperforms best triple (+17.4%).

Conclusions

This work presents an end-to-end pipeline for automated foot strike classification from trackside video. The best configuration achieves 76.7% test accuracy using only brightness/contrast augmentation. The 30-experiment augmentation study demonstrates that **targeted, domain-specific augmentation outperforms brute-force approaches**—a finding applicable to any fine-grained video classification task. The production-grade experiment infrastructure enabled reliable execution of 270+ GPU-hours with checkpointing, parallel execution, and cloud storage.

Future Work

- **More runners** (50+) to reduce overfitting
- **Regularisation**—dropout, weight decay, label smoothing, mixup
- **Cross-validation** across runner subsets
- **Larger models**—VideoMAE-Large (304M params) or VideoMAEv2
- **3-class classification** with sufficient "Toes first" data to separate it from flat foot

References

[1] Tong, Z., Song, Y., Wang, J., & Wang, L. (2022). VideoMAE: Masked Autoencoders are Data-Efficient Learners for Self-Supervised Video Pre-Training. *NeurIPS 2022*. https://arxiv.org/abs/2203.12602

[2] Dosovitskiy, A., et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR 2021*. https://arxiv.org/abs/2010.11929

[3] Ultralytics (2023). YOLOv8: Real-time Object Detection and Pose Estimation. https://github.com/ultralytics/ultralytics

[4] MCG-NJU/videomae-base-finetuned-kinetics. HuggingFace Model Hub. https://huggingface.co/MCG-NJU/videomae-base-finetuned-kinetics

[5] Moore, I. S., Ashford, K. J., Cross, C., Hope, J., Jones, H. S. R., & McCarthy-Ryan, M. (2022). Foot Strike Patterns During Overground Distance Running: A Systematic Review and Meta-Analysis. *Sports Medicine – Open*, 8, 82. https://link.springer.com/article/10.1186/s40798-021-00369-9

[6] Mumuni, A., & Mumuni, F. (2022). Data augmentation: A comprehensive survey of modern approaches. *Array*, 16, 100258. https://www.sciencedirect.com/science/article/pii/S2590005622000911

[7] Almeida, M. O., Davis, I. S., & Lopes, A. D. (2015). Biomechanical Differences of Foot-Strike Patterns During Running: A Systematic Review With Meta-analysis. *Journal of Orthopaedic & Sports Physical Therapy*, 45(10), 738–755. https://doi.org/10.2519/jospt.2015.6019